

Birdkit-Style Story Project: Build Spec for Claude

Oct 1, 2026 · @Justus Riley

How to use this doc

Give this doc to Claude (or paste the prompt in the last section) and it will scaffold a SvelteKit project whose build output matches the folder structure of a Birdkit-style NYT interactive: a prerendered story page, hashed `_app` code bundles and a separately deployed `_big_assets` media folder.

- **What you get:** a working source project, a build that emits the same output tree, placeholder content and the scroll mechanics from `scrolly-template.html`.
- **What it is not:** a copy of the NYT page. All copy, artwork, Lottie files and branding are placeholders you replace with your own. NYT's fonts (Cheltenham, Imperial) are licensed, so the spec uses free stand-ins.
- **Inferred vs. observed:** the output tree below was observed in the live page's network requests. The source tree is a reconstruction; Birdkit itself is private, so file names on the source side are conventions, not copies.
- **Run it in phases.** Each phase ends with a check you can run before moving on, matching the 10-chunk learning plan.

Target build output

The deploy folder must look like this after `npm run build`. Code and media live in two sibling folders, each with a content hash in its name so every file can be cached forever.

dist/	(served from <cdn>/projects/<project-
id>/)	
└ index.html	prerendered story page, full markup,
no JS required to read	
└ _app.<build-hash>/	
└ version.json	
└ immutable/	
└ entry/	
└ start.<hash>.js	SvelteKit client runtime
└ app.<hash>.js	app manifest and router

			└─ nodes/	
			└─ 0.<hash>.js	root layout
			└─ 1.<hash>.js	error page
			└─ 2.<hash>.js	the story page
			└─ chunks/<hash>.js (about 9)	shared code: Svelte runtime,
			components, Lottie wrapper	
			└─ assets/2.<hash>.css	all component CSS for the story page
			└─ _big_assets.<content-hash>/	
			└─ images/	
			└─ <name>.webp	two-up and painting images
			└─ slides/slide-1 ... slide-6/*.jpg	scroll section A artwork
			└─ videos/	
			└─ hero/hero.json	header Lottie
			└─ scrub-desktop.json	scroll-scrubbed Lottie, desktop
			└─ scrub-mobile.json	scroll-scrubbed Lottie, mobile
			└─ images/diagram/*.webp	image frames referenced by a Lottie
			file	
			└─ scripts/*.js	small standalone helpers

Rule	Why
<code>_app.<build-hash></code> changes on every build	New code never collides with cached old code
<code>_big_assets.<content-hash></code> changes only when media changes	Media uploads separately from code; a copy edit does not re-upload 100+ files
<code>index.html</code> is the only unhashed file	It is the one file you cache briefly or purge on deploy
Lottie JSON lives under <code>videos/</code>	Motion is shipped as data files, not code

Source project tree

One route, one content file, one component per block type. The `big_assets/` folder sits outside `src/` and `static/` so Vite never bundles it; a deploy script hashes and uploads it on its own.

```

my-story/
├─ package.json
├─ svelte.config.js          adapter-static, prerender on, custom
appDir
├─ vite.config.js
├─ .gitignore                dist/, .svelte-kit/, node_modules/
├─ README.md
├─ content/
│   └─ story.json            ordered blocks: text, two-up, diagram,
scrolly, lottie
├─ big_assets/              raw media, never imported by Vite
│   └─ images/
│       └─ slides/slide-1 ... slide-6/
│       └─ videos/
│           └─ hero/hero.json
│           └─ scrub-desktop.json
│           └─ scrub-mobile.json
│           └─ images/diagram/
│       └─ scripts/
├─ scripts/
│   └─ hash-assets.js        hashes big_assets/, copies to
dist/_big_assets.<hash>/, writes src/lib/assets.js
│   └─ deploy.js            uploads dist/ (optional; any static host
works)
├─ static/
│   └─ favicon.png
└─ src/
    └─ app.html              document shell; %sveltekit.head% and
%sveltekit.body%
    └─ app.css                tokens: gutter, column, type scale,
easing
    └─ routes/
        └─ +layout.svelte    imports app.css → becomes nodes/0
        └─ +layout.js        export const prerender = true
        └─ +error.svelte     → becomes nodes/1
        └─ +page.svelte      loops story.json into components →
becomes nodes/2
    └─ └─ +page.js            loads content/story.json at build time
        └─ lib/
            └─ assets.js      generated: ASSET_BASE =
'<cdn>/_big_assets.<hash>'
            └─ scroll.js      shared engine: progressOf(), rAF-
throttled listener

```

```

└─ lottie.js          loads lottie-web, play once or scrub by
progress
  └─ components/
    └─ Header.svelte   headline, dek, twin Lottie stage
    └─ Byline.svelte
    └─ Text.svelte     one <p class="g-text">
    └─ TwoUp.svelte
    └─ Diagram.svelte  grid of nodes + SVG connectors
    └─ Scrolly.svelte  runway + sticky panel + progress bar,
exposes step/progress via slot props
    └─ SlidesScrolly.svelte section A: hard-cut frames
    └─ CaptionScrolly.svelte section B: 65vh band + fading captions
    └─ PaintingsScrolly.svelte section C: per-step percent layouts
    └─ ScrubLottie.svelte  section D: desktop + mobile twins
    └─ Credits.svelte

```

How source maps to output: `+layout.svelte` compiles to `nodes/0`, `+error.svelte` to `nodes/1`, `+page.svelte` to `nodes/2`, and every component's scoped CSS merges into `assets/2.<hash>.css`. Shared imports (Svelte runtime, `scroll.js`, `lottie.js`) split into `chunks/`.

Config files

Three settings produce the output tree: `adapter-static` writes to `dist/`, `prerender` bakes the full markup into `index.html`, and a per-build `appDir` gives the `_app.<build-hash>` folder name.

package.json

```

{
  "name": "my-story",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite dev",
    "assets": "node scripts/hash-assets.js",
    "build": "npm run assets && vite build",
    "preview": "vite preview",
    "deploy": "node scripts/deploy.js"
  },
  "devDependencies": {
    "@sveltejs/adapter-static": "^3",

```

```

    "@sveltejs/kit": "^2",
    "@sveltejs/vite-plugin-svelte": "^5",
    "svelte": "^5",
    "vite": "^6"
  },
  "dependencies": {
    "lottie-web": "^5"
  }
}

```

svelte.config.js

```

import adapter from '@sveltejs/adapter-static';
import { createHash } from 'node:crypto';

// One hash per build; any unique string works (git SHA, timestamp).
const buildHash =
  createHash('sha256').update(String(Date.now())).digest('base64url').slice(0,
43);

export default {
  kit: {
    adapter: adapter({ pages: 'dist', assets: 'dist', fallback: undefined }),
    appDir: `_app.${buildHash}`,          // → dist/_app.<build-
hash>/immutable/...
    paths: { base: '', relative: true }, // relative URLs so the folder can
live under any CDN path
    prerender: { entries: ['*'] },
    output: { bundleStrategy: 'split' } // keeps entry/, nodes/, chunks/ as
separate files
  }
};

```

vite.config.js

```

import { sveltekit } from '@sveltejs/kit/vite';
import { defineConfig } from 'vite';

export default defineConfig({
  plugins: [sveltekit()],
  server: { fs: { allow: ['content', 'big_assets'] } } // dev server can read
story.json and media
});

```

src/routes/+layout.js

```
export const prerender = true;  
export const trailingSlash = 'never';
```

In dev, `ASSET_BASE` points at `/big_assets` so media loads without a build. The hash script swaps it to the hashed folder only for production builds.